



SISTEMAS OPERATIVOS - TEORÍA
17 de Enero de 2023

Nombre _____ DNI _____
Apellidos _____ Grupo _____

Cuestión 1. (2 puntos) Un sistema de ficheros UNIX utiliza bloques de 2048 bytes y punteros y direcciones de disco de 32 bits. Los nodos-i contienen 10 direcciones de disco para bloques de datos, una dirección de bloque índice indirecto simple y una dirección de bloque índice indirecto doble.

- a) ¿Cuál es el tamaño máximo de un fichero en este sistema?
- b) Si se reserva un bloque para el mapa de bits, ¿cuál es el tamaño máximo de la partición?
- c) Explique cuáles son las modificaciones al mapa de bits, tabla de nodos-i y bloques de datos cuando se ejecutan secuencialmente los siguientes comandos:

\$ mkdir /exam

\$ mv /home/kate/code.c /exam/

\$ ln -s /exam/code.c /home/jose/code.c

Cuestión 2. (2 puntos) Considere el siguiente módulo del kernel que implementa un driver de dispositivo de caracteres:

```
// ... includes go here ...
#define MAX_LEN 512
char message[MAX_LEN]="Hey!";

static int device_open(struct inode *inode, struct file *file) {
    try_module_get(THIS_MODULE);
    return 0;
}

static int device_release(struct inode *inode, struct file *file) {
    module_put(THIS_MODULE);
    return 0;
}

static ssize_t device_read(struct file *filp, char *buff,
                           size_t length, loff_t * offset) {
    int nr_bytes=MIN(length,strlen(message));

    if (copy_to_user(buff,message,nr_bytes))
        return -EFAULT;

    return nr_bytes;
}

static ssize_t device_write(struct file *filp, const char *buff,
                            size_t len, loff_t * offset) {
    int nr_bytes=MIN(length,MAX_LEN-1);

    if (copy_from_user(message,buff,nr_bytes))
        return -EFAULT;

    /* Close string */
    message[nr_bytes]='\0';

    (*offset)+=len; /* Update the file position indicator */
    printk(KERN_INFO "The message buffer was updated correctly\n")
    return len;
}

static struct file_operations fops = {
    .read = device_read,
    .write = device_write,
    .open = device_open,
    .release = device_release
};

// Implementation of init_module() and cleanup_module()
// ...
```

Asuma que (1) las funciones `init_module()` y `cleanup_module()` han sido correctamente implementadas, (2) que la estructura `fops` está bien asociada, (3) que tras la correcta carga del módulo, el kernel ha reservado para el driver el mayor number 248 y el minor number 0, y (4) que la implementación del comando `cat` en Linux usa un buffer de 1MiB para cada lectura con `read()`.

- a. Explique cuál es el propósito de la invocación de las funciones `copy_to_user()` y `copy_from_user()`. ¿Cuál es el significado del valor de retorno de dichas funciones?

- b. Proporcione un comando o secuencia de comandos para crear un fichero especial de caracteres `/dev/buffer` asociado al driver, y sobre el que cualquier usuario del sistema tenga permisos para leer y escribir.

- c. Suponga que el módulo del kernel acaba de cargarse en el núcleo, y que el fichero `/dev/buffer` se ha creado correctamente. En estas circunstancias, un usuario ejecuta el comando `cat /dev/buffer`. Responda razonadamente las siguientes preguntas: ¿Termina la ejecución de dicho comando correctamente? ¿Qué llamadas al sistema ejecuta el programa `cat` sobre el fichero `/dev/buffer` y cuántas veces se invoca cada una de estas llamadas? Ténganse en cuenta las llamadas al sistema que ejecutará el programa `cat` sobre el fichero `/dev/buffer` (que además desencadenen la invocación de funciones del código del driver) y sus valores de retorno, sabiendo que `cat` leerá del fichero hasta saber que ha llegado al final.

- d. Se desea que el driver tenga el siguiente comportamiento: que la lectura secuencial del fichero de dispositivo con el comando `cat` muestre el contenido de la última cadena de caracteres escrita en dicho fichero (p.ej. con `echo` y redirección), que no debe exceder el tamaño del array `message`. Ejemplo de ejecución:

```
$ echo Hello > /dev/buffer
$ cat /dev/buffer
Hello
$ echo Cool buffer > /dev/buffer
$ cat /dev/buffer
Cool buffer
```

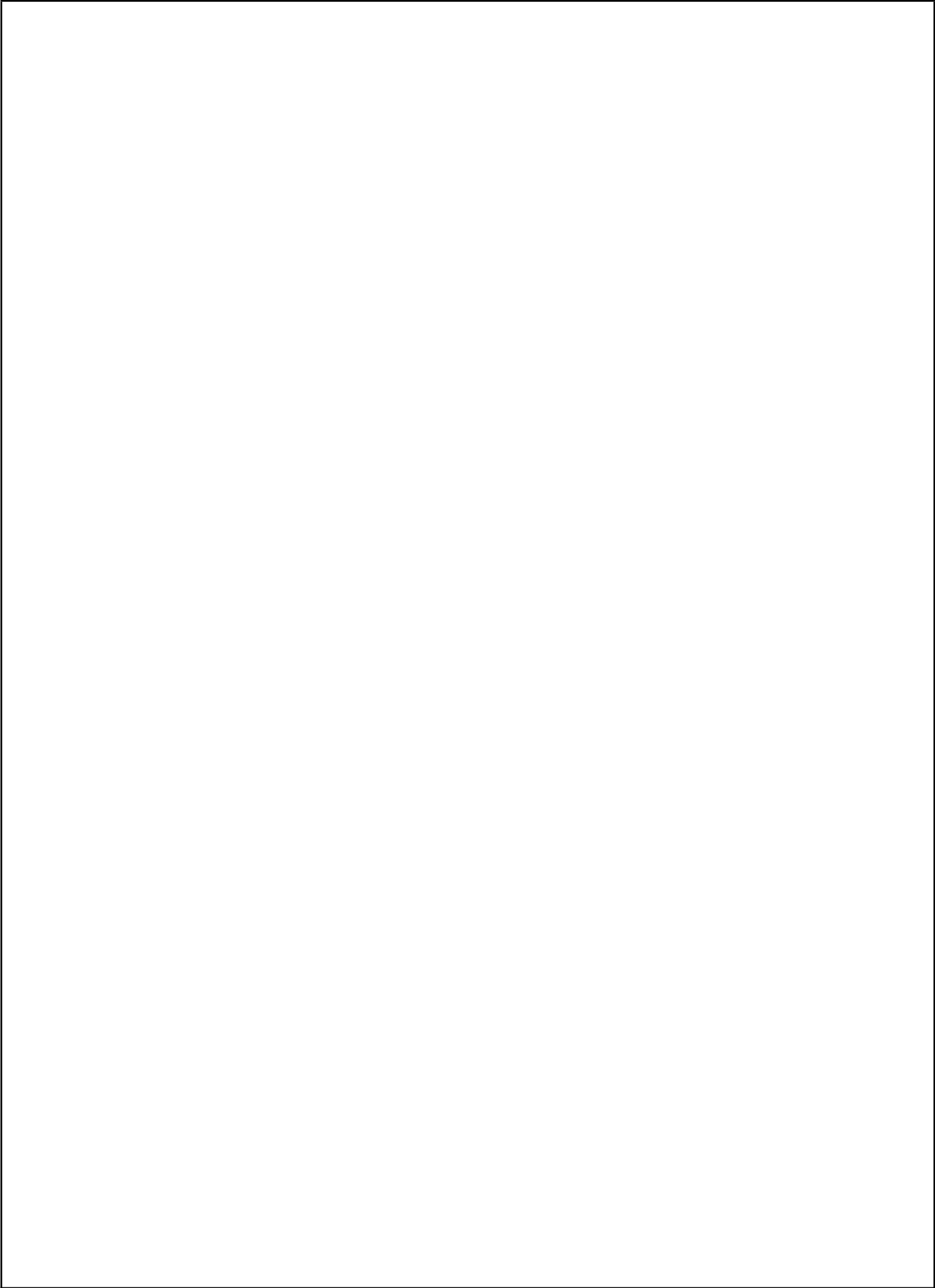
Realice las modificaciones mínimas necesarias en el código del driver para conseguir este comportamiento.

Cuestión 3. (2 puntos) Sincronización

Escribe un programa que cree cuatro hilos. Los tres primeros hilos tienen que imprimir por pantalla los números impares (del 1 al 9) con el mensaje: "threadX: numero", alternándose entre ellos en orden ascendente (primero lanzará hilo1, luego hilo2, después hilo 3). El cuarto hilo tiene que imprimir por pantalla los números pares (del 2 al 10) con el mensaje: "thread4: numero". El programa tiene que imprimir los números en orden creciente, de modo que los cuatro hilos se tendrán que alternar para conseguir un resultado como este:

```
thread1: 1
thread4: 2
thread2: 3
thread4: 4
thread3: 5
thread4: 6
thread1: 7
thread4: 8
thread2: 9
thread4: 10
```

Se recomienda utilizar mutex y variables de condición como mecanismo de sincronización.



Cuestión 4. (2 puntos) Un sistema utiliza gestión de memoria virtual con paginación bajo demanda con páginas de 4KB, enteros y direcciones de memoria de 32 bits. Se ejecuta en dicho sistema el siguiente programa:

```

/* 1 */ #include <stdio.h>
/* 2 */ #include <stdlib.h>
/* 3 */
/* 4 */ #define N 4092
/* 5 */
/* 6 */ typedef struct {
/* 7 */     char data[N];
/* 8 */     int key;
/* 9 */ } item;
/* 10 */
/* 11 */ item array[] = {
/* 12 */     {"bill", 3},
/* 13 */     {"neil", 4},
/* 14 */     {"john", 2},
/* 15 */     {"rick", 5},
/* 16 */     {"alex", 1},
/* 17 */ };
/* 18 */ item first;
/* 19 */

/* 20 */ void copy(item *a, item *b) {
/* 21 */     int i = 0;
/* 22 */
/* 23 */     for(; i < N; i++) {
/* 24 */         a->data[i] = b->data[i];
/* 25 */     }
/* 26 */     a->key = b->key;
/* 27 */ }
/* 28 */
/* 29 */ int main() {
/* 30 */     int j = 1;
/* 31 */     item * tmp, * pointer;
/* 32 */
/* 33 */     copy(&first,array);
/* 34 */     pointer = array;
/* 35 */     for(; j < 5; j++) {
/* 36 */         pointer++;
/* 37 */         tmp = malloc(sizeof(item));
/* 38 */         copy(tmp,pointer);
/* 39 */         printf("array[%d] = {%s, %d}\n",
/* 40 */             j, tmp->data, tmp->key);
/* 41 */     }
/* 42 */     free(tmp);
/* 43 */     return 0;
/* 44 */ }

```

Considerando que:

- Hay suficientes marcos de página libres en el sistema.
- El texto (código) del programa tras la compilación ocupa 1,5KB.
- Las bibliotecas son dinámicas, ocupan en total 4 páginas de código y 2 de datos y están residentes en memoria.
- El contenido inicial de la pila al lanzar a ejecución el programa ocupa 32 bytes.

a) Dadas las siguientes variables, indique en qué región del mapa de memoria están (cuando se cree dicha región durante la ejecución del proceso) y si tienen soporte en el ejecutable:

Variable	first	i	tmp	pointer	*tmp	*pointer
Región						
Soporte						

b) Identifique qué regiones lógicas (y su tamaño en páginas) constituyen la imagen de memoria del proceso que se crea al lanzar a ejecución dicho programa en las líneas 32 y 41.

- c) ¿Cuántos fallos de página producen los accesos a datos del programa y en qué momento de la ejecución (línea de código) tienen lugar?

- d) ¿Es correcto el comportamiento del programa desde el punto de vista de la gestión de memoria? Razona la respuesta.

Cuestión 5 (2 puntos) En un sistema GNU/Linux se ejecuta el siguiente programa:

```
#include <pthread.h>
#include <fcntl.h>
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#define MAX_THREADS 64

int fd=-1;

void* dump_string(void* arg){
    int count=(int) arg;
    char str[10];
    int j=0;
    int len=0;

    len=sprintf(str,"Thread%d\n",count);
    for (j=0;j<count;j++){
        write(fd,str,len);
        usleep(10);
    }
    return NULL;
}

int main(void){
    int i=0;
    int n=0;
    int times=3;
    pid_t pid;
    pid_t ancestor=getpid();
    pthread_t desc[MAX_THREADS];
    off_t pos;
    int len;
    char buf[10];
```

```
fd=open("fileout.txt",
        O_WRONLY|O_CREAT|O_TRUNC,0751);

while (times>0) {
    pid=fork();
    if (pid==0)
        write(fd,"Hi\n",3);
    n++;
    times--;
}

pos=lseek(fd,0,SEEK_CUR);
while (wait(NULL)!=-1){};

if (getpid()!=ancestor){
    close(fd);
    execl("/bin/echo","/bin/echo","Process",NULL);
    n++;
}

len=sprintf(buf,"%ld\n",pos);
write(fd,buf,len);

for (i=0;i<n;i++)
    pthread_create(&desc[i],NULL,
                  dump_string,i+1);

for (i=0;i<n;i++)
    pthread_join(desc[i],NULL);

close(fd);
exit(0);
}
```

Tenga en cuenta que la función `sprintf()` de la biblioteca estándar de C permite escribir un mensaje con formato (al estilo de `printf()`) pero volcando el contenido al buffer pasado como primer parámetro. La función retorna el número de caracteres escritos en el buffer.

Responda razonadamente las siguientes preguntas

- Indique cuántos procesos e hilos se crearán al ejecutar el programa (sin contar al padre original, que es el que invoca la función `main()`). ¿Cuántos procesos/hilos se ejecutarán de forma concurrente como máximo?

- b. Justifique si es posible determinar o no el orden en el que se ejecutarán los procesos e hilos creados. En caso afirmativo, indique cuál es el orden exacto de ejecución.

- c. Indique lo que el programa mostrará por pantalla. ¿Puede variar la salida mostrada en sucesivas ejecuciones del programa? En el caso de que haya varias opciones, proporcione dos salidas posibles.

- d. Especifique el contenido del fichero “fileout.txt” al finalizar la ejecución del programa. ¿Puede variar el contenido de este fichero en sucesivas ejecuciones del programa? En caso afirmativo, describa los posibles cambios que podrían observarse de una ejecución a otra, e indique dos posibles opciones para el contenido final del fichero.